Projeto e Análise de Algoritmos — Lista de Exercícios V

Instituto Federal de Minas Gerais, Campus Formiga · Bacharelado em Ciência da Computação · 2026/2 · Lista avaliada (5 pontos), entrega em 24/09/2026
Professor: Mário Luiz Rodrigues Oliveira · Aluna: Luisa Caetano

> **Como ler esta resolução.** Cada questão traz o enunciado literal, a solução em seções e, no fim, um quadro *Resposta*. Toda afirmação de Θ vem com c_1, c_2, n_0 numéricos, conferidos em $n = n_0$ exatamente. Ao citar o Teorema Mestre, "primeira/segunda/terceira linha" segue a ordem impressa no formulário da lista: $a < b^k$, $a = b^k$, $a > b^k$.

Questão 1 — máximo por divisão e conquista

> Enunciado
>
> "Considere o problema de encontrar o valor máximo em um vetor contendo n números reais. Projete um algoritmo usando o paradigma divisão e conquista para esse problema. Considere que n é um número par. Projete duas abordagens, conforme indicado:
> (a) (0.5 ponto) Projete um algoritmo para resolver o problema considerando um subproblema de tamanho n − 1.
> (b) (0.5 ponto) Projete um algoritmo para resolver o problema dividindo-o em dois subproblemas de tamanho n/2.
> (c) (0.5 ponto) Qual abordagem produz um algoritmo assintoticamente mais eficiente no pior caso? Justifique sua resposta. Respostas sem justificativas não serão consideradas."
>
> Formulário fornecido pela lista (Teorema Mestre, forma de Brassard & Bratley): sejam as constantes naturais a ≥ 1, b ≥ 2 e k ≥ 0; f(n) = c·n^k com c > 0 real; T(n) = aT(n/b) + f(n); n potência exata de b; T leva naturais em reais positivos e é assintoticamente não decrescente. Então T(n) ∈ θ(n^k) se a < b^k; θ(n^k·lg n) se a = b^k; θ(n^{log_b a}) se a > b^k.

0. Convenções

Vetor A[1..n] de reais, n ≥ 1. Custo medido em operações primitivas do modelo RAM (CLRS 3ª ed., §2.2): cada linha executada dos pseudocódigos abaixo, fora o tempo gasto dentro das chamadas recursivas, custa uma constante. Contam-se também, à parte, as **comparações entre elementos** do vetor, que é a medida usual para este problema.

Custo de uma chamada, fora as chamadas recursivas que ela faz. Cada algoritmo tem **as suas próprias** constantes — índice a para MaxA, índice b para MaxB; as de um não têm relação com as do outro:

– com tamanho 1 (caso base): uma constante $d_a > 0$ (MaxA) ou $d_b > 0$ (MaxB);
– com tamanho ≥ 2: depende do ramo tomado na combinação (o ramo "senão" executa uma linha a mais), mas fica sempre entre duas constantes, $0 < c'_a \le c''_a$ em MaxA e $0 < c'_b \le c''_b$ em MaxB, onde c″ é o custo quando o ramo "senão" é tomado.

Sobre "n é par": os dois algoritmos são escritos para **todo** n ≥ 1, e precisam ser. Em (a) o subproblema tem tamanho n − 1, que é ímpar; em (b), n par garante apenas que a **primeira** divisão é exatamente n/2 + n/2 — já com n = 6 os subproblemas têm tamanho 3, que se dividem em 2 + 1. Um algoritmo que só funcionasse para n par não se aplicaria aos próprios subproblemas.

(a) Um subproblema de tamanho n − 1

```
MaxA(A, n)                  ▷ devolve max{A[1], …, A[n]},  n ≥ 1
1  se n = 1 então devolva A[1]
2  m ← MaxA(A, n − 1)       ▷ conquista: máximo dos n − 1 primeiros
3  se m ≥ A[n] então devolva m
4  senão devolva A[n]       ▷ combina: compara com o último
```

Chamada inicial: MaxA(A, n).

Correção. P(n): "MaxA(A, n) devolve max{A[1..n]}", para todo n ≥ 1.
Base n = 1: a linha 1 devolve A[1] = max{A[1]}.
H.i.: seja n ≥ 2 e suponha P(n − 1) (note que n − 1 ≥ 1, então P(n − 1) está no domínio). Passo: pela h.i., a linha 2 obtém m = max{A[1..n−1]}; as linhas 3–4 devolvem o maior entre m e A[n], e max{A[1..n]} = max(max{A[1..n−1]}, A[n]). Logo P(n). ∎

Custo. Uma execução de MaxA(A, n) faz n chamadas no total (a inicial e n − 1 recursivas, com tamanhos n, n − 1, …, 1): uma com tamanho 1, de custo d_a, e n − 1 com tamanho ≥ 2, cada uma de custo entre c'_a e c''_a. Somando chamada a chamada, **toda** entrada de tamanho n custa entre $d_a + c'_a(n-1)$ e $d_a + c''_a(n-1)$.
(Com um custo constante c_a por chamada de tamanho ≥ 2 — $c_a = c''_a$ no pior caso —, isso é a recorrência $T_a(1) = d_a$, $T_a(n) = T_a(n-1) + c_a$ para n ≥ 2, cuja forma fechada é $T_a(n) = d_a + c_a(n-1)$: n = 1 dá d_a ✓; se vale para n − 1, $T_a(n) = d_a + c_a(n-2) + c_a = d_a + c_a(n-1)$ ✓.)
Pior caso. O vetor estritamente crescente toma o ramo "senão" (linha 4) em todas as n − 1 combinações, pois ali A[n] é maior que todos os anteriores; seu custo é $d_a + c''_a(n-1)$, que é também a cota superior de qualquer entrada. Portanto o custo de pior caso é exatamente

$$T_a^{\text{pior}}(n) = d_a + c''_a \cdot (n-1).$$

$\Theta(n)$ com $c_1 = \min(c''_a, d_a)$, $c_2 = c''_a + d_a$, $n_0 = 1$:
– cota inferior: $d_a + c''_a(n-1) \ge \min(c''_a,d_a)\cdot 1 + \min(c''_a,d_a)\cdot(n-1) = \min(c''_a,d_a)\cdot n$;
– cota superior: $d_a + c''_a(n-1) \le d_a \cdot n + c''_a \cdot n = (c''_a + d_a)\cdot n$, pois n ≥ 1. Em n = n_0 = 1: $\min(c''_a,d_a) \le d_a \le c''_a + d_a$ ✓. (As mesmas contas com c'_a mostram que até o melhor caso é Θ(n).)

(Este algoritmo não é da forma aT(n/b) + f(n): o Teorema Mestre não se aplica; resolve-se por expansão, como acima.)
Comparações entre elementos: $C_a(1) = 0$ e $C_a(n) = C_a(n-1) + 1$, logo $\mathbf{C_a(n) = n - 1}$ — esta contagem não depende dos valores do vetor.
Chamadas (contando a inicial): n. Profundidade da pilha: n.

(b) Dois subproblemas de tamanho n/2

```
MaxB(A, i, j)                  ▷ devolve max{A[i], …, A[j]},  i ≤ j
1  se i = j então devolva A[i]
2  q ← ⌊(i + j)/2⌋             ▷ divide ao meio
3  esq ← MaxB(A, i, q)         ▷ conquista a metade esquerda
4  dir ← MaxB(A, q + 1, j)     ▷ conquista a metade direita
5  se esq ≥ dir então devolva esq
6  senão devolva dir           ▷ combina
```

Chamada inicial: MaxB(A, 1, n).

Tamanhos. Seja $s = j - i + 1 \geq 2$. A metade esquerda tem $q - i + 1 = \lfloor (j-i)/2 \rfloor + 1 = \lfloor (s-1)/2 \rfloor + 1 = \lceil s/2 \rceil$ elementos; a direita tem $s - \lceil s/2 \rceil = \lfloor s/2 \rfloor$. Para $s \geq 2$ os dois são ≥ 1 e $< s$, então a recursão termina. Com $s = n$ par, os dois têm exatamente $n/2$, como pede o enunciado.

Correção. Indução forte em s. P(s): "MaxB devolve o máximo de qualquer trecho de s elementos", para todo $s \geq 1$. Base $s = 1$: linha 1. H.i.: seja $s \geq 2$ e suponha P(t) para todo $1 \leq t < s$. Passo: os tamanhos $\lceil s/2 \rceil$ e $\lfloor s/2 \rfloor$ estão em $[1, s-1]$, então pela h.i. esq = máximo da metade esquerda e dir = máximo da metade direita; o máximo do trecho é o maior dos dois, que as linhas 5–6 devolvem. Logo P(s). ∎

Custo. Primeiro, um fato sobre a recorrência
$R(1) = u$, $R(s) = R(\lceil s/2 \rceil) + R(\lfloor s/2 \rfloor) + v$ para $s \geq 2$,
com u, v reais quaisquer: sua solução é **exata para todo $s \geq 1$** (não é preciso supor potência de 2):
$R(s) = u \cdot s + v \cdot (s-1)$.
Prova por indução forte: $s = 1$ dá u ✓; para $s \geq 2$, pela h.i.,
$R(s) = u\lceil s/2 \rceil + v(\lceil s/2 \rceil - 1) + u\lfloor s/2 \rfloor + v(\lfloor s/2 \rfloor - 1) + v = u \cdot s + v(s-2) + v = u \cdot s + v(s-1)$ ✓, usando $\lceil s/2 \rceil + \lfloor s/2 \rfloor = s$ (a indução não usa o sinal de u nem de v).
A árvore de chamadas de MaxB depende só de s, não dos valores do vetor. O número de chamadas de tamanho 1 satisfaz R com (u, v) = (1, 0), e o de chamadas de tamanho ≥ 2 satisfaz R com (u, v) = (0, 1): são s e $s - 1$ ($2s - 1$ no total). Somando chamada a chamada, toda entrada de tamanho n custa entre $d_b \cdot n + c'_b(n-1)$ e $d_b \cdot n + c''_b(n-1)$. Com um custo constante c_b por chamada de tamanho ≥ 2 ($c_b = c''_b$ no pior caso), o custo é $T_b(1) = d_b$,
$T_b(s) = T_b(\lceil s/2 \rceil) + T_b(\lfloor s/2 \rfloor) + c_b$, isto é, R com $(u, v) = (d_b, c_b)$:
$T_b(s) = d_b \cdot s + c_b \cdot (s-1)$.
Pior caso. No vetor estritamente crescente, em toda combinação o máximo da metade direita é maior que o da esquerda, e o ramo "senão" (linha 6) é tomado em todas as $n - 1$ combinações; o custo é $d_b \cdot n + c''_b(n-1)$, a cota superior. Portanto
$T_b^{\text{pior}}(n) = d_b \cdot n + c''_b \cdot (n-1)$.
$\Theta(n)$ com $c_1 = d_b$, $c_2 = c''_b + d_b$, no = 1: $d_b \cdot n \leq d_b \cdot n + c''_b(n-1) \leq (c''_b + d_b) \cdot n$ para $n \geq 1$. Em n = 1: $d_b \leq d_b \leq c''_b + d_b$ ✓.

Confirmação pelo Teorema Mestre (n potência exata de 2, onde a recorrência vira $T_b(n) = 2T_b(n/2) + c_b = 2T_b(n/2) + c_b \cdot n^0$; aqui a, b, k são os parâmetros do formulário, e c_b faz o papel do seu c): $a = 2$, $b = 2$, $k = 0$, $c_b > 0$; $b^k = 2^0 = 1 < 2 = a$, isto é, $a > b^k$ — **terceira linha** do formulário: $T_b(n) \in \Theta(n^{\log_2 2}) = \Theta(n)$. As hipóteses sobre T valem: $T_b(s) = (c_b + d_b)s - c_b \geq d_b > 0$ é positiva e crescente em s. Para n par que não é potência de 2 vale a forma fechada exata acima, que dá a mesma classe.
Comparações entre elementos: $C_b(1) = 0$, $C_b(s) = C_b(\lceil s/2 \rceil) + C_b(\lfloor s/2 \rfloor) + 1$, logo $\mathbf{C_b(n) = n - 1}$ (R com (u, v) = (0, 1)) — também independente dos valores.
Chamadas (contando a inicial): $2n - 1$. Profundidade da pilha: $1 + \lceil \lg n \rceil$.

(c) Qual é assintoticamente mais eficiente no pior caso?

Nenhuma das duas. Ambas levam tempo **$\Theta(n)$** no pior caso:
$T_a^{\text{pior}}(n) = d_a + c''_a(n-1) \in \Theta(n)$ e $T_b^{\text{pior}}(n) = d_b \cdot n + c''_b(n-1) \in \Theta(n)$, com as testemunhas exibidas em (a) e (b). Como as duas estão em $\Theta(n)$, vale $T_a^{\text{pior}} \in \Theta(T_b^{\text{pior}})$: nenhuma é assintoticamente melhor. (As constantes de uma e de outra não são comparáveis entre si — cada uma depende das linhas do seu pseudocódigo —, e a pergunta é sobre a classe.) O desvio que depende dos dados (linhas 3–4 em MaxA, 5–6 em MaxB) só altera o custo de cada chamada dentro de uma faixa constante; por isso até o melhor caso de cada uma é $\Theta(n)$ — a classe não depende da entrada, só as constantes.

Mais do que isso: as duas fazem **exatamente o mesmo número de comparações entre elementos, $n - 1$**, qualquer que seja a entrada. A divisão ao meio não reduz trabalho — ela só organiza o mesmo número de comparações numa árvore balanceada em vez de numa cadeia. Em chamadas (contando a inicial), (b) até faz mais: $2n - 1$ contra n — diferença de constante, não de classe.

Por que dividir ao meio não dá lg n aqui. A intuição da busca binária engana: lá há **um** subproblema de tamanho n/2 ($a = 1$, $T(n) = T(n/2) + \Theta(1) \in \Theta(\lg n)$); aqui há **dois** ($a = 2$). Para n potência de 2, o nível i ($0 \leq i < \lg n$) da árvore de recursão de MaxB tem 2^i chamadas internas, cada uma com uma combinação de custo constante, e o nível lg n tem as n folhas, que não combinam: o número de combinações dobra de um nível para o seguinte enquanto o tamanho cai pela metade, e a soma de 2^i para $i = 0, \ldots, \lg n - 1$ dá $n - 1$ (soma vazia se $n = 1$). Para n qualquer, a contagem $n - 1$ vem da recorrência de C_b acima.

Nenhuma abordagem pode ser melhor que $\Theta(n)$. Considere qualquer algoritmo determinístico correto que acessa o vetor só lendo posições, ao custo de ao menos uma operação por leitura. Ele precisa ler as n posições: se numa entrada ele nunca lê A[p], troque A[p] por um valor maior que o valor devolvido e que todos os demais elementos; a execução não muda (A[p] não é lido), a saída não muda, e agora está errada. Logo todo algoritmo gasta ao menos $1 \cdot n$ operações para todo $n \geq 1$, isto é, $\Omega(n)$, e as duas abordagens são ótimas.

A diferença que existe é de espaço, não de tempo: a pilha de recursão tem profundidade n em (a) e $1 + \lceil \lg n \rceil$ em (b). Com A passado por referência e cada quadro de pilha de tamanho constante, isso é $\Theta(n)$ contra $\Theta(\lg n)$ de memória auxiliar; para o segundo,
$\lg n \leq 1 + \lceil \lg n \rceil \leq 2 \lg n$ para todo $n \geq 2$ ($c_1 = 1$, $c_2 = 2$, $n_0 = 2$): a cota inferior vale porque $\lceil \lg n \rceil \geq \lg n$; a superior, para $n \geq 4$, porque $1 + \lceil \lg n \rceil < 2 + \lg n \leq 2 \lg n$, e em $n = 2$ e $n = 3$ por conta direta ($2 \leq 2$; $3 \leq 2 \lg 3 \approx 3{,}17$). $n_0 = 1$ não serve, pois $\lg 1 = 0$ e a profundidade em $n = 1$ é 1.

Resposta

(a) MaxA: resolve A[1..n−1] recursivamente e compara o resultado com A[n].
Recorrência $T_a(1) = d_a$, $T_a(n) = T_a(n-1) + c_a$; pior caso (vetor estritamente crescente, $c_a = c''_a$): $T_a^{\text{pior}}(n) = d_a + c''_a(n-1) \in \Theta(n)$; $n - 1$ comparações em toda entrada.

(b) MaxB: divide em duas metades ($\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$; exatamente $n/2$ e $n/2$ na primeira divisão, pois n é par), resolve cada uma e devolve o maior dos dois resultados.
$T_b(n) = 2T_b(n/2) + c_b \Rightarrow \Theta(n)$ pelo Teorema Mestre (n potência de 2; $a = 2 > b^k = 1$, terceira linha); para todo n, pior caso exatamente $T_b^{\text{pior}}(n) = d_b \cdot n + c''_b(n-1) \in \Theta(n)$; $n - 1$ comparações em toda entrada.

(c) Nenhuma é assintoticamente mais eficiente: ambas são $\Theta(n)$ no pior caso (e mesmo no melhor), com exatamente $n - 1$ comparações; e $\Theta(n)$ é ótimo, pois todo algoritmo precisa ler as n posições. A única diferença assintótica é a profundidade da pilha: $\Theta(n)$ em (a), $\Theta(\lg n)$ em (b).

Questão 2 — busca num vetor sem fim

> Enunciado
>
> "Resolva o seguinte problema:
> a) (0.5 ponto) Dado um vetor A [.] de tamanho infinito no qual as primeiras m posições contêm números inteiros em ordem crescente e as demais posições são preenchidas com o valor ∞. Não se conhece o valor de m. Descreva um algoritmo que recebe um valor inteiro x como entrada e encontra uma posição no vetor contendo o valor x, se tal posição existir, em tempo o(n). (Se você não se sente confortável com o fato do vetor ter comprimento infinito, assuma em vez disso comprimento de tamanho n, contudo você não conhece esse comprimento. Assuma, também, que a implementação da estrutura de dados vetor em sua linguagem de programação preferida retorna a mensagem de erro ∞ sempre que elementos A[i] com i > n são acessados).
> b) (0.5 ponto) Argumente que seu algoritmo tem a complexidade solicitada."

0. Modelo

Posições 1, 2, 3, …; A[1..m] são inteiros em ordem crescente (o argumento abaixo vale também para ordem não decrescente, com repetições); A[i] = ∞ para i > m; $m \geq 0$ é desconhecido. O valor ∞ é tratado como maior que todo inteiro, então a sequência inteira A[1], A[2], … é não decrescente:

(Monotonia) $i < i' \Rightarrow A[i] \leq A[i']$.

Na versão de comprimento finito n (desconhecido), as posições $m < i \leq n$ contêm ∞ e o acesso a $i > n$ devolve ∞; o algoritmo enxerga exatamente a mesma sequência que na versão infinita, e $m \leq n$.
Custo: cada acesso A[i] e cada operação aritmética ou comparação custa $\Theta(1)$ (modelo RAM); conta-se o número de acessos ao vetor, e o tempo é no máximo uma constante vezes esse número.

Tamanho da entrada. Na versão infinita o único parâmetro é m, e o(n) se lê com $n = m$; na versão finita, n é o comprimento e $m \leq n$. Define-se

$f(m) = 2\lfloor \lg m \rfloor + 3$ para $m \geq 1$, $f(0) = 2$.

f é não decrescente em $m \geq 0$ ($f(0) = 2 < 3 = f(1)$, e $\lfloor \lg m \rfloor$ não decresce), de modo que $f(m) \leq f(n)$ em qualquer das leituras, sempre que $m \leq n$.

(a) O algoritmo: galope seguido de busca binária

```
BuscaIlimitada(A, x)
 1  i ← 1
 2  enquanto A[i] < x faça        ▷ fase 1 (galope): testa as posições 1, 2, 4, 8, …
 3      i ← 2·i
 4  lo ← ⌊i/2⌋ + 1;  hi ← i        ▷ se i = 1, então lo = hi = 1
 5  enquanto lo ≤ hi faça          ▷ fase 2: busca binária em A[lo..hi]
 6      meio ← ⌊(lo + hi)/2⌋;  v ← A[meio]
 7      se v = x então devolva meio
 8      se v < x então lo ← meio + 1
 9      senão hi ← meio − 1
10  devolva "x não está no vetor"
```

A ideia: não se conhece o fim da parte útil, então se procura, dobrando o índice, a primeira potência de 2 cujo conteúdo já é ≥ x. Isso delimita um trecho onde x tem de estar, e nele se faz a busca binária usual. Nenhum passo usa m ou n. Na fase 2 cada volta lê o vetor **uma** vez (linha 6) e guarda o valor em v.

(b) Correção e complexidade

Lema 1 (a fase 1 termina, e quantas voltas dá). Seja J o número de vezes que o corpo da linha 3 executa. Então $i = 2^J$ ao fim da fase 1, e

$J = 0$ se $m = 0$; $J \le \lfloor \lg m \rfloor + 1$ se $m \ge 1$.

Prova. O teste A[i] < x só é verdadeiro se A[i] é finito, isto é, $i \le m$ (pois $\infty < x$ é falso). Logo, cada vez que o corpo executa, o valor corrente $i = 2^t$ satisfaz $2^t \le m$, com t = 0, 1, 2, … em sequência. Se m = 0, A[1] = ∞ e o laço não executa: J = 0. Se m ≥ 1, os valores de t com $2^t \le m$ são 0, 1, …, $\lfloor \lg m \rfloor$ — no máximo $\lfloor \lg m \rfloor + 1$ deles —, e portanto o corpo executa no máximo $\lfloor \lg m \rfloor + 1$ vezes; em particular a fase 1 sempre termina. ∎

Lema 2 (invariante ao fim da fase 1). Ao fim da fase 1, A[i] ≥ x (o laço parou) e, se J ≥ 1, A[i/2] < x (o teste anterior foi verdadeiro, na posição $i/2 = 2^{J-1}$).

Lema 3 (x, se existe, está em A[lo..hi]). Se existe p com A[p] = x, então existe uma tal posição em [lo, hi].

Prova. Se A[i] = x, a própria posição i = hi serve. Se A[i] > x, seja p qualquer posição com A[p] = x. Para p ≥ i, a Monotonia daria A[p] ≥ A[i] > x; logo p < i = hi. Se J ≥ 1, para p ≤ i/2 a Monotonia daria A[p] ≤ A[i/2] < x; logo p > i/2, isto é, $p \ge \lfloor i/2 \rfloor + 1 =$ lo. Se J = 0, lo = 1 ≤ p trivialmente. ∎

Lema 4 (busca binária). Invariante do laço 5–9: "toda posição de [loo, hio] com valor x está em [lo, hi]", onde [loo, hio] é o trecho inicial. Vale no início. Se v = A[meio] < x, toda posição p ≤ meio tem A[p] ≤ A[meio] < x, então descartar [lo, meio] preserva o invariante; se v > x, toda posição p ≥ meio tem A[p] > x, e descartar [meio, hi] o preserva. Logo, se o laço termina sem devolver, [lo, hi] ficou vazio e x não ocorre em [loo, hio] — e, pelo Lema 3, não ocorre no vetor. Reciprocamente, o algoritmo só devolve meio quando A[meio] = x. As posições com ∞ dentro do trecho não atrapalham: ∞ > x, e o trecho continua ordenado.

Número de voltas: se o trecho tem L ≥ 1 posições, após uma volta sem retorno ele tem no máximo $\lfloor L/2 \rfloor$ (a esquerda [lo, meio − 1] tem $\lfloor (L-1)/2 \rfloor \le \lfloor L/2 \rfloor$ posições; a direita [meio + 1, hi] tem $\lceil (L-1)/2 \rceil = \lfloor L/2 \rfloor$). Depois de r voltas, no máximo $\lfloor L/2^r \rfloor$; como uma volta só começa com trecho não vazio, r voltas executadas implicam $L/2^{r-1} \ge 1$, isto é, $r \le \lfloor \lg L \rfloor + 1$. ∎

Correção. Pelos Lemas 3 e 4, o algoritmo devolve uma posição p com A[p] = x quando x ocorre no vetor, e responde que x não está quando não ocorre. ∎

Contagem de acessos — cota superior. Fase 1: J + 1 acessos (J testes verdadeiros e o falso que encerra). Fase 2: o trecho tem L = hi − lo + 1 = i − $\lfloor i/2 \rfloor$ posições, isto é, $L = 2^{J-1}$ se J ≥ 1 e L = 1 se J = 0; pelo Lema 4, no máximo $\lfloor \lg L \rfloor + 1$ voltas, com **uma** leitura por volta (linha 6): no máximo J leituras se J ≥ 1, e 1 leitura se J = 0. Total:

$J = 0$: no máximo $1 + 1 = 2$ acessos;

$J \ge 1$: no máximo $(J + 1) + J = 2J + 1 \le 2(\lfloor \lg m \rfloor + 1) + 1 = 2\lfloor \lg m \rfloor + 3$ (Lema 1). Como $J \ge 1$ exige $m \ge 1$, conclui-se: **para todo m ≥ 0, no máximo f(m) acessos** ($f(m) = 2\lfloor \lg m \rfloor + 3$ para m ≥ 1, e f(0) = 2). O tempo total é no máximo K·f(m) para uma constante K > 0 (cada volta de cada laço faz Θ(1) operações além do acesso), isto é, O(lg m) para m ≥ 2. A cota é atingida quando $m = 2^t - 1$ e x > A[m] (conferido por força bruta para m ≤ 699: atingida em m = 1, 3, 7, 15, …, 511).

Contagem de acessos — cota inferior (pior caso). Seja m ≥ 1 e tome x = A[m] + 1 (inteiro, maior que todo elemento finito). Então $A[2^t] < x$ para toda posição $2^t \le m$, e o teste da linha 2 é verdadeiro exatamente para t = 0, 1, …, $\lfloor \lg m \rfloor$: $J = \lfloor \lg m \rfloor + 1$, e só a fase 1 já faz $J + 1 = \lfloor \lg m \rfloor + 2$ acessos. Logo o pior caso W(m) (máximo de acessos sobre as entradas com esse m) satisfaz

$\lfloor \lg m \rfloor + 2 \le W(m) \le 2\lfloor \lg m \rfloor + 3$ para todo m ≥ 1.

Testemunhas de $W \in \Theta(\lg m)$: $c_1 = 1$, $c_2 = 5$, $m_0 = 2$. A cota inferior vale porque $\lfloor \lg m \rfloor + 2 > \lg m$; a superior, porque $2\lfloor \lg m \rfloor + 3 \le 2 \lg m + 3 \lg m = 5 \lg m$ quando $\lg m \ge 1$, isto é, m ≥ 2. Em $m = m_0 = 2$: lg 2 = 1, e $1 \le W(2) \le 5$ (de fato W(2) = 4). $m_0 = 1$ não serve: lg 1 = 0.

Por que isso é o(n). Pela definição (CLRS 3ª ed., §3.1), $g(n) \in o(n)$ se para **toda** constante c > 0 existe $n_0 > 0$ tal que $0 \le g(n) < c \cdot n$ para todo $n \ge n_0$.

Lema 5. $2^j \ge j^2$ para todo inteiro j ≥ 4.

Base j = 4: 16 ≥ 16. Passo: se $2^j \ge j^2$ com j ≥ 4, então $2^{j+1} = 2 \cdot 2^j \ge 2j^2$, e

$2j^2 - (j+1)^2 = j^2 - 2j - 1 = (j-1)^2 - 2 \ge 9 - 2 > 0$; logo $2^{j+1} \ge (j+1)^2$. ∎

(Falha em j = 3: 8 < 9 — por isso a restrição.)

Consequência: para n ≥ 16, $j = \lfloor \lg n \rfloor \ge 4$ e $n \ge 2^j \ge j^2$, donde $\lfloor \lg n \rfloor \le \sqrt{n}$. Então, para n ≥ 16,

$f(n) = 2\lfloor \lg n \rfloor + 3 \le 2\sqrt{n} + 3\sqrt{n} = 5\sqrt{n}$ (pois $\sqrt{n} \ge 1$).
Dado $c > 0$, tome $n_0 = \max(16, \lfloor 25/c^2 \rfloor + 1)$. Para $n \ge n_0$ vale $n > 25/c^2$, isto é, $\sqrt{n} > 5/c$; multiplicando por $c\cdot\sqrt{n} > 0$: $c\cdot n > 5\sqrt{n} \ge f(n)$. Então $0 \le f(n) < c\cdot n$ para todo $n \ge n_0$, e $f \in o(n)$. Para o tempo $K\cdot f(n)$, repita com c/K no lugar de c: $n_0 = \max(16, \lfloor 25K^2/c^2 \rfloor + 1)$. Exemplo em $n = n_0$: $c = 1$ dá $n_0 = 26$, $f(26) = 11 < 26$; $c = 1/10$ dá $n_0 = 2501$, $f(2501) = 25 < 250{,}1$.
Na versão finita ($n \ge 1$), o tempo é $\le K\cdot f(m) \le K\cdot f(n)$ (f não decrescente, $m \le n$), portanto também $o(n)$.

Por que as alternativas óbvias não servem. A busca linear lê A[1], A[2], … até achar x ou um valor > x; com $x > A[m]$ ela lê $m + 1$ posições — $\Theta(m)$ no pior caso, que **não** é $o(n)$ (na leitura $n = m$, ou na versão finita com $m = n$). A busca binária direta precisa do extremo direito do trecho, que é justamente o que não se conhece. O galope descobre um extremo válido: $h_i = 2^J$, com $h_i \le 2m$ quando $J \ge 1$ (pois $2^{J-1} \le m$) e $h_i = 1$ quando $J = 0$ — em no máximo $\lfloor \lg m \rfloor + 2$ acessos se $m \ge 1$, e 1 acesso se $m = 0$.

Resposta

(a) Galope + busca binária: teste A[1], A[2], A[4], …, $A[2^t]$, … até achar a primeira posição $i = 2^J$ com $A[i] \ge x$ (as posições com ∞ garantem a parada); então faça busca binária em $A[\lfloor i/2 \rfloor + 1 \,..\, i]$, lendo A[meio] uma vez por volta. Devolve uma posição com valor x, ou "não está".

(b) A fase 1 dá $J \le \lfloor \lg m \rfloor + 1$ dobras (só se dobra enquanto $A[i]$ é finito, isto é, $i \le m$), e a fase 2 busca num trecho de 2^{J-1} posições (1 posição se $J = 0$), com no máximo J leituras (1 se $J = 0$). Total: no máximo $2\lfloor \lg m \rfloor + 3$ acessos para $m \ge 1$ (2 para $m = 0$); como x maior que todos força $\lfloor \lg m \rfloor + 2$ acessos, o pior caso é $\Theta(\lg m)$ ($c_1 = 1$, $c_2 = 5$, $m_0 = 2$), com $m \le n$. E $2\lfloor \lg n \rfloor + 3 < c\cdot n$ para $n \ge \max(16, \lfloor 25/c^2 \rfloor + 1)$, qualquer que seja $c > 0$ — logo o tempo é $o(n)$.

Questão 3 — qual dos algoritmos A, B, C

Enunciado

"Suponha que, para um problema com entradas de tamanho n, você tenha que escolher um dentre três algoritmos A, B e C. Considere que trata-se do pior caso de cada algoritmo.
1. O algoritmo A resolve problemas dividindo-os em cinco subproblemas de tamanho n/2, recursivamente resolve cada subproblema e então combina as soluções em tempo linear. 2. O algoritmo B resolve problemas de tamanho n, recursivamente resolvendo dois subproblema [sic] de tamanho $n - 1$ e então combina as soluções em tempo constante.
3. O algoritmo C resolve problemas dividindo-os em nove subproblemas de tamanho n/3, recursivamente resolve cada subproblema e então combina as soluções em tempo $\theta(n^2)$. Indique qual dos algoritmos é assintoticamente mais eficiente no pior caso. Expresse sua resposta em termos da notação θ. Justifique sua resposta. Resposta sem justificativa não será considerada."

Formulário fornecido pela lista (Teorema Mestre, forma de Brassard & Bratley): sejam as constantes naturais $a \ge 1$, $b \ge 2$ e $k \ge 0$; $f(n) = c\cdot n^k$ com $c > 0$ real; $T(n) = aT(n/b) + f(n)$; n potência exata de b; T leva naturais em reais positivos e é assintoticamente não decrescente. Então $T(n) \in \theta(n^k)$ se $a < b^k$; $\theta(n^k \cdot \lg n)$ se $a = b^k$; $\theta(n^{\log_b a})$ se $a > b^k$. A lista também fornece a hierarquia $1 < \log n < \sqrt{n} < n < n\cdot\log n < n^c < c^n < n! < n^n$, "onde c é uma constante arbitrária inteira ≥ 2". Ela não cobre diretamente as comparações desta questão ($\log_2 5$ não é inteiro, e em $n^c < c^n$ a base é o mesmo c do expoente); por isso a §5 as prova diretamente.

0. As três recorrências

Casos base: $T(1) = t > 0$ (qualquer constante positiva). O custo de combinar é positivo:
A: $T_A(n) = 5\cdot T_A(n/2) + f_A(n)$, $f_A(n) \in \Theta(n)$ ("tempo linear")
B: $T_B(n) = 2\cdot T_B(n - 1) + f_B(n)$, $f_B(n) \in \Theta(1)$ ("tempo constante")
C: $T_C(n) = 9\cdot T_C(n/3) + f_C(n)$, $f_C(n) \in \Theta(n^2)$
Primeiro resolve-se cada uma com f exatamente da forma $c\cdot n^k$, $c > 0$ (a que o teorema exige); depois, na §4, mostra-se que trocar "$= c\cdot n^k$" por "$\in \Theta(n^k)$" não muda a classe.

Hipótese de monotonia (a última do formulário, "T assintoticamente não decrescente"), para quaisquer $t, c > 0$. Nas potências de b, onde a recorrência define T: $T(b\cdot n) = a\cdot T(n) + f(b\cdot n) > T(n)$, pois $a \ge 1$, $T(n) > 0$ e $f(b\cdot n) > 0$; e $T(1) = t > 0$, então por indução T é positiva e crescente sobre as potências de b. Para B, do mesmo modo, $T_B(n) = 2T_B(n - 1) + c > T_B(n - 1) > 0$. Que o custo não diminua com n a partir de algum N (isto é, $T(n) \le T(n + 1)$ para todo $n \ge N$) é o que "T assintoticamente não decrescente" significa; o formulário o assume, é natural para tempo de pior caso, e é usado na Observação da §5.

1. Algoritmo A — Teorema Mestre, terceira linha

$T_A(n) = 5T_A(n/2) + c\cdot n$: $a = 5$, $b = 2$, $k = 1$, $c > 0$. $b^k = 2^1 = 2$ e $a = 5 > 2$: **terceira linha** do formulário ($a > b^k$). Logo
$T_A(n) \in \Theta(n^{\log_2 5})$, com $\log_2 5 \approx 2{,}3219$.
Forma fechada ($n = 2^m$, $T(1) = t$): $T_A(n) = (t + 2c/3)\cdot n^{\log_2 5} - (2c/3)\cdot n$.
Checagem com $t = c = 1$: $T_A(n) = (5/3)\cdot n^{\log_2 5} - (2/3)\cdot n$; $T_A(2) = 5\cdot 1 + 2 = 7$ e $(5/3)\cdot 5 - (2/3)\cdot 2 = 25/3 - 4/3 = 7$ ✓.
Testemunhas (t, c quaisquer): $c_1 = t$, $c_2 = t + 2c/3$, $n_0 = 1$. A cota superior é imediata; a inferior, $(t + 2c/3)n^{\log_2 5} - (2c/3)n \ge t\cdot n^{\log_2 5}$, equivale a $(2c/3)(n^{\log_2 5} - n) \ge 0$, verdadeira para $n \ge 1$ porque $\log_2 5 > 1$. Com $t = c = 1$: $c_1 = 1$, $c_2 = 5/3$, e em $n = n_0 = 1$: $1 \le 1 \le 5/3$ ✓.

2. Algoritmo B — fora do Teorema Mestre

A chamada é T(n − 1), não T(n/b): subtrair 1 não é dividir por uma constante b (n − 1 = n/b exigiria b = n/(n − 1), que varia com n). O teorema não se aplica; resolve-se por expansão. Com $T_B(1) = t$ e custo de combinação constante $f_B(n) = c > 0$:

$T_B(n) = 2^{n-1}\cdot t + c\cdot(2^{n-1} - 1) = 2^{n-1}\cdot(t + c) - c$ para $n \geq 1$.

Prova por indução: n = 1 dá t + c − c = t ✓; se vale para n − 1 (n ≥ 2),
$2\cdot[2^{n-2}(t + c) - c] + c = 2^{n-1}(t + c) - c$ ✓.

Testemunhas: $(t/2)\cdot 2^n = 2^{n-1}\cdot t \leq T_B(n) \leq 2^{n-1}(t + c) = ((t + c)/2)\cdot 2^n$ para n ≥ 1, isto é $c_1 = t/2$, $c_2 = (t + c)/2$, $n_o = 1$. Com t = c = 1: $T_B(n) = 2^n - 1$, $c_1 = 1/2$, $c_2 = 1$, e em n = 1: $1 \leq 1 \leq 2$ ✓. Logo $\mathbf{T_B(n) \in \Theta(2^n)}$.

(Intuição: a árvore de recursão é binária completa de altura n − 1, com $2^n - 1$ nós.)

3. Algoritmo C — Teorema Mestre, segunda linha

$T_C(n) = 9T_C(n/3) + c\cdot n^2$: a = 9, b = 3, k = 2, c > 0. $b^k = 3^2 = 9 = a$: **segunda linha** ($a = b^k$). Logo

$\mathbf{T_C(n) \in \Theta(n^2 \cdot \lg n)}$.

Forma fechada ($n = 3^m$, T(1) = t): $T_C(n) = n^2\cdot(t + c\cdot\log_3 n)$ — cada um dos m níveis da árvore custa $c\cdot 9^i\cdot(3^{m-i})^2 = c\cdot 9^m = c\cdot n^2$, mais as n^2 folhas de custo t.

Checagem com t = c = 1: $T_C(n) = n^2(\log_3 n + 1)$; $T_C(3) = 9 + 9 = 18 = 9\cdot(1 + 1)$ ✓. Testemunhas (t, c quaisquer): $c_1 = c/\lg 3$, $c_2 = (t + c)/\lg 3$, $n_o = 3$. Como $\log_3 n = \lg n / \lg 3$ e, para $n \geq 3$, $1 \leq \lg n / \lg 3$: $(c/\lg 3)\cdot n^2 \lg n \leq n^2(t + c\log_3 n) \leq ((t + c)/\lg 3)\cdot n^2 \lg n$. Com t = c = 1: $c_1 = 1/\lg 3$, $c_2 = 2/\lg 3$; em $n = n_o = 3$, $n^2 \lg n = 9 \lg 3$, logo $c_1\cdot 9 \lg 3 = 9 \leq 18 \leq 18 = c_2\cdot 9 \lg 3$ ✓ (igualdade na cota superior).

$n_o = 1$ não serve: $\lg 1 = 0$ e $T_C(1) = t > 0$.

4. De "$= c\cdot n^k$" para "$\in \Theta(n^k)$"

O enunciado diz "tempo linear", "tempo constante" e "$\theta(n^2)$", não exatamente $c\cdot n^k$. Isso não muda nenhuma classe, pelo seguinte lema de comparação.

Lema. Sejam T e T′ definidas pela mesma recorrência (mesmo a, mesma forma de chamada), com o mesmo valor no caso base e com $f(n) \leq f'(n)$ para todo n. Então $T(n) \leq T'(n)$ para todo n. Prova: indução sobre n (nas potências de b, ou em n ≥ 1 para B), usando a > 0: $T(n) = a\cdot T(\cdot) + f(n) \leq a\cdot T'(\cdot) + f'(n) = T'(n)$. ∎

Como $f(n) \in \Theta(n^k)$ e $f(n) > 0$ para todo n (combinar custa alguma coisa), existem c′, c″ > 0 com $c'\cdot n^k \leq f(n) \leq c''\cdot n^k$ para **todo** $n \geq 1$: para $n \geq n_o$ isso é a definição de Θ, e para os finitos $n < n_o$ basta diminuir c′ e aumentar c″ ($f(n)/n^k$ é positivo em cada um deles). Pelo Lema, T fica entre as soluções com $f = c'\cdot n^k$ e com $f = c''\cdot n^k$, e as duas estão na mesma classe Θ (§1–§3, cujas formas fechadas e testemunhas valem para quaisquer t, c > 0). Logo $T_A \in \Theta(n^{\log_2 5})$, $T_B \in \Theta(2^n)$, $T_C \in \Theta(n^2 \lg n)$.

5. Comparação

C é assintoticamente mais eficiente que A. $n^2 \lg n / n^{\log_2 5} = \lg n / n^\varepsilon$ com $\varepsilon = \log_2 5 - 2 = \log_2(5/4) \approx 0{,}3219 > 0$. Aplicando L'Hôpital às funções de variável real x > 0 (o limite da sequência é o mesmo),
$\lim \lg x / x^\varepsilon = \lim (1/(x \ln 2)) / (\varepsilon\cdot x^{\varepsilon-1}) = \lim 1/(\varepsilon\cdot \ln 2\cdot x^\varepsilon) = 0$.
Então $n^2 \lg n \in o(n^{\log_2 5})$: para quaisquer constantes positivas c_2^C e c_1^A, a partir de algum n vale $c_2^C\cdot n^2 \lg n < c_1^A\cdot n^{\log_2 5}$, e portanto — com as duas cotas valendo no mesmo n, pela Observação abaixo — $T_C(n) < T_A(n)$.

A é assintoticamente mais eficiente que B. $n^{\log_2 5} \leq n^3$ para n ≥ 1, e $n^3/2^{n^A} \to 0$ (três aplicações de L'Hôpital, de novo em variável real: $x^3/2^x \to 3x^2/(2^x \ln 2) \to 6x/(2^x \ln^2 2) \to 6/(2^x \ln^3 2) \to 0$, cada seta significando "tem o mesmo limite que"). Logo $n^{\log_2 5} \in o(2^n)$. Ordem: **C ≺ A ≺ B**, isto é, $\Theta(n^2 \lg n) \prec \Theta(n^{\log_2 5}) \prec \Theta(2^n)$.

Observação — comparar A (definido nas potências de 2) com C (potências de 3) no mesmo n. O teorema é enunciado para n potência exata de b; é a hipótese "T assintoticamente não decrescente" que permite estender a conclusão a todo n. Esboço: se T é não decrescente para $n \geq N$ e $c_1 g(b^j) \leq T(b^j) \leq c_2 g(b^j)$ para $b^j \geq n_o$, então para $b^j \leq n < b^{j+1}$ (com $b^j \geq \max(n_o, N, b)$) vale
$T(n) \leq T(b^{j+1}) \leq c_2 g(b^{j+1}) \leq c_2 g(b\cdot n)$ e $T(n) \geq T(b^j) \geq c_1 g(b^j) \geq c_1 g(n/b)$, usando que g é não decrescente para n ≥ 1; para $g(n) = n^e\cdot \lg^p n$ (e > 0, p ≥ 0 — é o caso de $n^{\log_2 5}$ e de $n^2 \lg n$) tem-se $g(b\cdot n) \leq b^e\cdot 2^p\cdot g(n)$ e $g(n/b) \geq g(n)/(b^e\cdot 2^p)$ para $n \geq b^2$, de modo que a classe Θ(g) vale para todo n grande. Com isso, T_A e T_C são comparáveis no mesmo n.

A vantagem de C é assintótica: para as funções de referência (sem as constantes ocultas, que não se conhecem), a razão $\lg n / n^{0{,}3219}$ ainda vale ≈ 1,07 em $n = 2^{10}$ e cai para ≈ 0,23 em $n = 2^{20}$.

Resposta

A: $T(n) = 5T(n/2) + \Theta(n) \Rightarrow a = 5 > b^k = 2$ (terceira linha) $\Rightarrow \Theta(n^{\log_2 5})$, com $\log_2 5 \approx 2{,}32$.

B: $T(n) = 2T(n - 1) + \Theta(1) \Rightarrow$ fora do teorema (n − 1 não é n/b); com T(1) = t e custo de combinação constante c, a expansão dá $T(n) = 2^{n-1}(t + c) - c$, e pelo lema da §4 qualquer custo em Θ(1) fica entre duas dessas $\Rightarrow \Theta(2^n)$.

C: $T(n) = 9T(n/3) + \Theta(n^2) \Rightarrow a = 9 = b^k = 9$ (segunda linha) $\Rightarrow \Theta(n^2 \lg n)$.

O mais eficiente no pior caso é **C, com $\Theta(n^2 \lg n)$**; a ordem é C ≺ A ≺ B, porque $n^2 \lg n \in o(n^{\log_2 5})$ ($\log_2 5 > 2$ e $\lg n \in o(n^\varepsilon)$ para $\varepsilon > 0$) e $n^{\log_2 5} \in o(2^n)$.

Questão 4 — prova por indução

> Enunciado
> "(1.0 ponto) Prove a assertiva abaixo usando o método da indução matemática.
> a) $1/2 + 1/4 + 1/8 + \cdots + 1/2^n < 1$, $\forall n \geq 1$"

Notação

Para cada inteiro $n \geq 1$, $S(n) = \sum_{i=1}^{n} 1/2^i$. A escrita $1/2 + 1/4 + 1/8 + \ldots + 1/2^n$ do enunciado é a notação usual para essa soma: só entram os termos com $i \leq n$. Para $n \leq 2$ a forma com reticências é apenas ilustrativa — vale a definição por $\sum$: $S(1) = 1/2$ (um único termo) e $S(2) = 1/2 + 1/4 = 3/4$.

Por que a indução ingênua (separando o último termo) não fecha

Tentando provar Q(n): $S(n) < 1$ separando o **último** termo, o passo seria
$S(n+1) = S(n) + 1/2^{n+1} < 1 + 1/2^{n+1}$ — e isso não é < 1. Usando só a informação "$S(n) < 1$", o passo exigiria a implicação, para todo real s, "se $s < 1$ então
$s + 1/2^{n+1} < 1$", que é falsa (contraexemplo da implicação, não da assertiva: $s = 31/32$ e $n = 3$ dão $31/32 + 1/16 = 33/32 \geq 1$). A hipótese "$S(n) < 1$" perde a informação de **quanto** falta para 1. Há duas saídas: fortalecer a hipótese (Prova 1) ou separar o **primeiro** termo em vez do último (Prova 2). As duas são provas completas; basta uma.

Prova 1 — fortalecendo a hipótese

Proposição. Para todo inteiro $n \geq 1$, P(n): $S(n) = 1 - 1/2^n$.

Base ($n = 1$). Lado esquerdo: $S(1) = 1/2$. Lado direito: $1 - 1/2^1 = 1/2$. Iguais: P(1) vale.

Hipótese de indução. Seja $n \geq 1$ um inteiro fixado e suponha P(n): $S(n) = 1 - 1/2^n$.

Passo. Separando o último termo da soma até $n + 1$:

$$\begin{aligned} S(n+1) &= S(n) + 1/2^{n+1} \\ &= 1 - 1/2^n + 1/2^{n+1} \leftarrow \text{h.i. aplicada a } S(n) \\ &= 1 - 2/2^{n+1} + 1/2^{n+1} \\ &= 1 - 1/2^{n+1}, \end{aligned}$$

que é P(n+1). Como P(1) vale e P(n) ⇒ P(n+1) para todo $n \geq 1$, pelo princípio da indução P(n) vale para todo $n \geq 1$.

Conclusão. Como $2^n > 0$, tem-se $1/2^n > 0$ e $S(n) = 1 - 1/2^n < 1$ para todo $n \geq 1$. ∎

Prova 2 — a própria assertiva como hipótese de indução, Q(n), separando o primeiro termo

Proposição. Para todo inteiro $n \geq 1$, Q(n): $S(n) < 1$.

Base ($n = 1$). $S(1) = 1/2 < 1$.

Hipótese de indução. Seja $n \geq 1$ um inteiro fixado e suponha Q(n): $S(n) < 1$.

Passo. Separando o primeiro termo e pondo $1/2$ em evidência no restante:

$$\begin{aligned} S(n+1) &= 1/2 + \sum_{i=2}^{n+1} 1/2^i = 1/2 + \sum_{j=1}^{n} 1/2^{j+1} = 1/2 + (1/2)\cdot\sum_{j=1}^{n} 1/2^j \\ &= 1/2 + S(n)/2 \\ &< 1/2 + 1/2 = 1 \leftarrow \text{h.i.: } S(n) < 1, \text{ logo } S(n)/2 < 1/2 \end{aligned}$$

(divide-se por $2 > 0$) isto é, Q(n+1). Como Q(1) vale e Q(n) ⇒ Q(n+1) para todo $n \geq 1$, pelo princípio da indução Q(n) vale para todo $n \geq 1$. ∎
(A troca de índice é $j = i - 1$, que leva $i = 2, \ldots, n+1$ em $j = 1, \ldots, n$; com $n = 1$ as duas somas têm um único termo, 1/4.)

Conferência

$S(1) = 1/2$, $S(2) = 3/4$, $S(3) = 7/8 = 1 - 1/8$. A identidade $S(n) = 1 - 1/2^n$, a desigualdade $S(n) < 1$ e a relação $S(n+1) = 1/2 + S(n)/2$ foram conferidas em aritmética racional exata para $n = 1, \ldots, 399$.

Resposta

Por indução em n, prova-se a forma mais forte P(n): $1/2 + 1/4 + \ldots + 1/2^n = 1 - 1/2^n$ (base: $1/2 = 1/2$; passo: $1 - 1/2^n + 1/2^{n+1} = 1 - 1/2^{n+1}$); daí a soma é < 1, pois $1/2^n > 0$. A indução direta em "< 1" separando o último termo não fecha; ela fecha separando o primeiro: $S(n+1) = 1/2 + S(n)/2 < 1/2 + 1/2 = 1$.

Questão 5 — Teorema Mestre em dez recorrências

> Enunciado
>
> "(0.5 ponto) Usando o teorema mestre, indique o limite assintótico usando a notação Θ, para cada uma das equações de recorrência abaixo. Considere o caso base T(1) = 1 ou T(n) = c, para alguma constante c e ∀n ≤ n0, para alguma constante n0. Suponha, também, que n uma [sic] potência exata de b. Escolha o valor adequado para b conforme a equação de recorrência a ser resolvida. Justifique todas as suas respostas. Respostas sem justificativas não serão consideradas. Pode acontecer de não ser aplicável a versão do teorema mestre discutida em sala de aula e também apresentada nessa lista de exercícios (veja em Informações que pode [sic] ser úteis). Nesse(s) caso(s) apresente o motivo pelo qual essa versão do teorema mestre não se aplica.
> (a) T(n) = T(n/3) + 1
> (b) T(n) = 2T(n/2) + n ∗ lgn
> (c) T(n) = T(2n/3) + n
> (d) T(n) = 5T(n/4) − n
> (e) T(n) = T(n − 1) + 3
> (f) T(n) = T(n/3) + 0
> (g) T(n) = 5T(n/4) − n²
> (h) T(n) = T(2n/4) + n
> (i) T(n) = 3/2T(n/4) − n²
> (j) T(n) = 3T(n/2) + n"
>
> Teorema mestre (literal, das "Informações que podem ser úteis"): "Sejam as constantes naturais a ≥ 1, b ≥ 2 e k ≥ 0. Considere a função f(n) da forma $c * n^k$, na qual c > 0 e ∈ R, seja a equação de recorrência dada por T(n) = aT(n/b) + f(n) e suponha n uma potência exata de b. Considere ainda que T é uma função que leva números naturais em números reais positivos e que T é assintoticamente não decrescente, Nessas condições, [BRASSARD and BRATLEY, 1996] indicam que a solução dessa recorrência é limitada assintoticamente por T(n) = $\theta(n^k)$ se a < b^k; $\theta(n^k * \lg n)$ se a = b^k; $\theta(n^{\log_b a})$ se a > b^k."

0. O que o teorema exige — a lista de conferência

Para aplicar esta versão é preciso ter, **todas ao mesmo tempo**:

(H1) a forma T(n) = a·T(n/b) + f(n) — uma única chamada "dividida por b", repetida a vezes;
(H2) a natural, a ≥ 1;
(H3) b natural, b ≥ 2;
(H4) f(n) = c·n^k exatamente, com c > 0 real e k ≥ 0 natural;
(H5) T com valores reais positivos e assintoticamente não decrescente (conferida abaixo nas potências de b, que é onde a recorrência define T).

Os casos, **na ordem em que o formulário os imprime**:

1ª linha: a < b^k → Θ(n^k); 2ª linha: a = b^k → Θ(n^k lg n); 3ª linha: a > b^k → Θ($n^{\log_b a}$). Nos itens aplicáveis confere-se o resultado com a forma fechada exata (T(1) = 1, n = b^m) e exibem-se testemunhas c_1, c_2, no verificadas em n = no. Nos itens não aplicáveis dá-se o motivo e, à parte, o que a recorrência realmente faz — o que mostra que a hipótese violada não é detalhe: em (f), (g) e (i), aplicar a fórmula às cegas daria resposta **errada** qualquer que seja o caso base; em (d), daria resposta errada com o caso base T(1) = 1 do enunciado (e com qualquer T(1) ≤ 4).

(a) T(n) = T(n/3) + 1 — APLICA

a = 1, b = 3, f(n) = 1 = 1·n^0 ⇒ c = 1 > 0, k = 0. (H1)–(H4) valem.
b^k = 3^0 = 1 = a: **2ª linha** ⇒ T(n) ∈ Θ(n^0·lg n) = **Θ(lg n)**.
Forma fechada (n = 3^m, T(1) = 1): T(3^m) = m + 1, isto é, T(n) = $\log_3$ n + 1. Positiva e crescente nas potências de 3: (H5) ✓.
Testemunhas: c_1 = 1/lg 3, c_2 = 2/lg 3, no = 3. Com $\log_3$ n = lg n / lg 3 e 1 ≤ lg n / lg 3 para n ≥ 3: (1/lg 3)·lg n ≤ $\log_3$ n + 1 ≤ (2/lg 3)·lg n. Em n = no = 3: 1 ≤ T(3) = 2 ≤ 2 ✓. no = 1 não serve: lg 1 = 0 e T(1) = 1 > c_2·0.

(b) T(n) = 2T(n/2) + n·lg n — NÃO APLICA

a = 2, b = 2 servem, mas **f(n) = n lg n não é da forma c·n^k** (viola H4). Não existem c, k com n lg n = c·n^k para todo n potência de 2: n = 2 daria 2 = c·2^k e n = 4 daria 8 = c·4^k, donde 2^k = 4, k = 2 e c = 1/2; mas em n = 8, c·8^k = 32 ≠ 24 = 8·lg 8. Nem mesmo "f ∈ Θ(n^k)" salvaria: para k ∈ {0, 1}, n lg n / n^k ≥ lg n → ∞; para k ≥ 2, n lg n / n^k ≤ lg n / n → 0. Então f não está em Θ(n^k) para k natural nenhum.
À parte (fora do teorema): por expansão, T(2^m) = 2^m·(1 + m(m + 1)/2), isto é, T(n) = n·(1 + lg n·(lg n + 1)/2) ∈ Θ(n $\lg^2$ n) (c_1 = 1/2, c_2 = 2, no = 2; em n = 2: 1 ≤ 4 ≤ 4).

(c) T(n) = T(2n/3) + n — NÃO APLICA

2n/3 = n/(3/2): o "b" seria **3/2, que não é natural nem ≥ 2** (viola H3). Também não há "n potência exata de b" inteira: $(3/2)^m$ = $3^m/2^m$, com numerador ímpar e denominador par, não é inteiro para m ≥ 1.
À parte: com piso, T(n) = T(⌊2n/3⌋) + n e T(1) = 1, vale n ≤ T(n) ≤ 3n (soma geométrica de razão 2/3), logo Θ(n) — conferido para n = 1…200 000.

(d) T(n) = 5T(n/4) − n — NÃO APLICA

a = 5, b = 4, mas f(n) = −n = (−1)·n^1: **c = −1 < 0** (viola H4). Com o caso base T(1) = 1, (H5) também falha: T(4) = 5 − 4 = 1 e T(16) = 5 − 16 = −11 < 0 — T deixa de ser positiva. À parte: com a recorrência aplicada desde n = 4, T(4^m) = (T(1) − 4)·$n^{\log_4 5}$ +

4n. A resposta dependeria do caso base: $\Theta(n^{\log_4 5})$ se T(1) > 4; exatamente 4n se T(1) = 4; negativa a partir de algum n se T(1) < 4. Com a outra opção do enunciado, T(n) = C para n ≤ n0 (chama-se aqui de C a constante do caso base, que o enunciado chama de c, para não confundir com o coeficiente c = −1 de f), o limiar muda: sendo 4^{m0} a maior potência de 4 que é ≤ n0 (n0 ≥ 1), $T(4^m) = ((C - 4\cdot 4^{m0})/5^{m0})\cdot n^{\log_4 5} + 4n$ para m ≥ m0; o sinal do coeficiente líder é o de $C - 4\cdot 4^{m0}$: positivo dá $\Theta(n^{\log_4 5})$, zero dá exatamente 4n, negativo faz T ficar negativa (por exemplo, n0 = 4 e C = 10 dão $T(4^6) < 0$). Aplicar o teorema às cegas (a = 5 > 4 = b^k, 3ª linha) daria $\Theta(n^{\log_4 5})$ — errado com T(1) = 1, quando T é negativa; só coincidiria com o comportamento real quando T(1) > 4 (ou $C > 4\cdot 4^{m0}$).

(e) T(n) = T(n − 1) + 3 — NÃO APLICA

A chamada é T(n − 1), não T(n/b) (viola H1): n − 1 = n/b exigiria b = n/(n − 1), que varia com n (vale 2 só em n = 2 e fica entre 1 e 2 para n ≥ 3) — não é uma constante. É uma recorrência subtrativa.
À parte: por expansão, T(n) = T(1) + 3(n − 1) = 3n − 2 (com T(1) = 1) ∈ Θ(n); $c_1 = 1$, $c_2 = 3$, $n_0 = 1$ ($n \le 3n - 2 \iff n \ge 1$; em n = 1: 1 ≤ 1 ≤ 3).

(f) T(n) = T(n/3) + 0 — NÃO APLICA

a = 1, b = 3, mas f(n) = 0 **não é da forma $c\cdot n^k$ com c > 0** (viola H4): $0 = c\cdot n^k$ obriga c = 0. A exigência não é formalidade: aplicando a fórmula com "c = 0, k = 0" (a = 1 = b^0, 2ª linha) sairia Θ(lg n), que é **falso**.
À parte: $T(3^m) = T(1)$ para todo m — T é constante, Θ(1) ($c_1 = c_2 = 1$, $n_0 = 1$, com T(1) = 1).

(g) T(n) = 5T(n/4) − n² — NÃO APLICA

a = 5, b = 4, mas f(n) = $-n^2$: **c = −1 < 0** (viola H4). E T não é positiva (viola H5): com T(1) = 1, T(4) = 5 − 16 = −11.
À parte: $T(4^m) = (T(1) + 16/11)\cdot n^{\log_4 5} - (16/11)\cdot n^2$. Como $\log_4 5 \approx 1{,}161 < 2$, o termo $-(16/11)n^2$ domina e T fica negativa a partir de algum n **qualquer que seja o caso base** (com T(1) = 1 já em n = 4; com T(1) = 10^6, em n = 4^{12}); com T(n) = C para n ≤ n0 muda só o coeficiente de $n^{\log_4 5}$, e o de n^2, −16/11, não depende do caso base. Às cegas (a = 5 < 16 = b^k, 1ª linha) sairia $\Theta(n^2)$ — absurdo para uma função que tende a $-\infty$.

(h) T(n) = T(2n/4) + n — APLICA

Simplificando o argumento: 2n/4 = n/2, logo **b = 2** (natural ≥ 2). a = 1, $f(n) = n = 1\cdot n^1 \Rightarrow c = 1 > 0$, k = 1. (H1)–(H4) valem.
$b^k = 2^1 = 2 > 1 = a$, isto é, $a < b^k$: **1ª linha** ⇒ T(n) ∈ **Θ(n)**.
Forma fechada (n = 2^m, T(1) = 1): T(n) = 2n − 1 (T(2) = 1 + 2 = 3 ✓). Positiva e crescente: (H5) ✓.
Testemunhas: $c_1 = 1$, $c_2 = 2$, $n_0 = 1$: $n \le 2n - 1 \iff n \ge 1$, e 2n − 1 ≤ 2n. Em n = 1: 1 ≤ 1 ≤ 2 ✓. A armadilha do item: ler "2n/4" e escolher b = 4 (ou a = 2). O valor adequado de b sai do argumento já simplificado.

(i) T(n) = (3/2)·T(n/4) − n² — NÃO APLICA

Lendo "3/2T(n/4)" como (3/2)·T(n/4): **a = 3/2 não é natural** (viola H2) e **c = −1 < 0** (viola H4) — dois motivos independentes. (Na outra leitura, 3/(2·T(n/4)), nem a forma a·T(n/b) + f(n) existe — H1; e em qualquer leitura f(n) = $-n^2$ tem c < 0. O veredito não depende da leitura.) E T não é positiva (H5): com T(1) = 1, T(4) = 3/2 − 16 = −29/2.
À parte: $T(4^m) = (T(1) + 32/29)\cdot(3/2)^m - (32/29)\cdot n^2$, negativa a partir de algum n para qualquer caso base, pois $(3/2)^m = n^{\log_4(3/2)}$ e $\log_4(3/2) \approx 0{,}29 < 2$ (com T(n) = C para n ≤ n0 muda só o coeficiente de $(3/2)^m$; o de n^2, −32/29, não depende do caso base). Às cegas (a = 3/2 < 16 = b^k, 1ª linha) sairia $\Theta(n^2)$ — absurdo para uma função que tende a $-\infty$.

(j) T(n) = 3T(n/2) + n — APLICA

a = 3, b = 2, $f(n) = n = 1\cdot n^1 \Rightarrow c = 1 > 0$, k = 1. (H1)–(H4) valem.
$b^k = 2 < 3 = a$: **3ª linha** ⇒ T(n) ∈ **$\Theta(n^{\log_2 3})$**, com $\log_2 3 \approx 1{,}585$.
Forma fechada (n = 2^m, T(1) = 1): $T(n) = 3\cdot n^{\log_2 3} - 2n$, isto é, $T(2^m) = 3^{m+1} - 2^{m+1}$ (T(2) = 3 + 2 = 5 = 9 − 4 ✓). Positiva e crescente: $T(2^{m+1}) - T(2^m) = 2\cdot 3^{m+1} - 2^{m+1} > 0$. Testemunhas: $c_1 = 1$, $c_2 = 3$, $n_0 = 1$: a cota inferior $3n^{\log_2 3} - 2n \ge n^{\log_2 3}$ equivale a $n^{\log_2 3 - 1} \ge 1$, verdadeira para n ≥ 1. Em n = 1: 1 ≤ 1 ≤ 3 ✓.

O caso base não muda as classes dos itens aplicáveis

Com T(1) = t > 0 em vez de 1: (a) T(n) = $\log_3 n$ + t; (h) T(n) = 2n − 2 + t; (j) $T(n) = (t + 2)\cdot n^{\log_2 3} - 2n$. Com T(n) = C para n ≤ n0 (a outra opção do enunciado, com a constante do caso base chamada de C como em (d); C > 0, pois o teorema pede T positiva, e n0 ≥ 1), sendo b^{m0} a maior potência de b que é ≤ n0: (a) $T(3^m) = C + (m - m0)$; (h) $T(2^m) = 2n + C - 2^{m0+1}$;
(j) $T(2^m) = ((C + 2\cdot 2^{m0})/3^{m0})\cdot n^{\log_2 3} - 2n$, para m ≥ m0. Em todos os casos o termo dominante tem coeficiente positivo e as classes Θ(lg n), Θ(n) e $\Theta(n^{\log_2 3})$ se mantêm; só as constantes c_1, c_2 (e o no das testemunhas) mudam.

Quadro-resumo

item	a	b	f(n)	hipótese violada	resposta
(a)	1	3	$1 = n^0$	—	$a = b^k = 1$, 2ª linha: $\Theta(\lg n)$
(b)	2	2	$n \lg n$	H4: não é $c \cdot n^k$	não aplica (de fato $\Theta(n \lg^2 n)$)
(c)	1	3/2	n	H3: $b = 3/2$	não aplica (de fato $\Theta(n)$)
(d)	5	4	$-n$	H4: $c < 0$; H5 com $T(1) = 1$	não aplica (T fica negativa com $T(1) = 1$)
(e)	—	—	3	H1: $T(n-1)$	não aplica (de fato $\Theta(n)$; $3n - 2$ com $T(1) = 1$)
(f)	1	3	0	H4: $c = 0$	não aplica (de fato $\Theta(1)$, não $\Theta(\lg n)$)
(g)	5	4	$-n^2$	H4: $c < 0$; H5	não aplica ($T \to -\infty$ para todo caso base)
(h)	1	2	n	—	$a = 1 < b^k = 2$, 1ª linha: $\Theta(n)$
(i)	3/2	4	$-n^2$	H2: $a = 3/2$; H4: $c < 0$; H5	não aplica ($T \to -\infty$ para todo caso base)
(j)	3	2	n	—	$a = 3 > b^k = 2$, 3ª linha: $\Theta(n^{\log_2 3})$

Resposta

(a) $\Theta(\lg n)$ — $a = 1$, $b = 3$, $k = 0$, $a = b^k$.
(b) Não se aplica: $f(n) = n \lg n$ não é da forma $c \cdot n^k$.
(c) Não se aplica: $b = 3/2$ não é natural ≥ 2.
(d) Não se aplica: $c = -1 < 0$ (e, com $T(1) = 1$, T fica negativa: $T(16) = -11$).
(e) Não se aplica: $T(n-1)$ não é da forma $T(n/b)$ (recorrência subtrativa).
(f) Não se aplica: $f(n) = 0$ exige $c = 0$, e o teorema pede $c > 0$ (a fórmula daria $\Theta(\lg n)$; o certo é $\Theta(1)$).
(g) Não se aplica: $c = -1 < 0$ (e T fica negativa: $T(4) = -11$ com $T(1) = 1$).
(h) $\Theta(n)$ — $2n/4 = n/2$, logo $a = 1$, $b = 2$, $k = 1$, $a < b^k$.
(i) Não se aplica: $a = 3/2$ não é natural, e $c = -1 < 0$.
(j) $\Theta(n^{\log_2 3})$ — $a = 3$, $b = 2$, $k = 1$, $a > b^k$.

Informações que podem ser úteis

Anexo da lista (transcrição literal)

Somatórios e Logaritmos:

- $\Sigma_{i=1}^{n} (i) = n(n+1)/2$
- $\Sigma_{i=1}^{n} (a^i) = a(a^n - 1)/(a - 1)$
- $\log (a * b) = \log a + \log b$
- $\log (a/b) = \log a - \log b$
- $\log_b a = \log_c a / \log_c b$
- $\log_b a^c = c * \log_b a$
- $\lg a = \log_2 a$

Hierarquia de funções:

A seguinte hierarquia de funções pode ser definida do ponto de vista assintótico:

$$1 < \log(n) < \sqrt{n} < n < n * \log(n) < n^c < c^n < n! < n^n$$

onde c é uma constante arbitrária inteira ≥ 2.

Teorema mestre:

Sejam as constantes naturais $a \geq 1$, $b \geq 2$ e $k \geq 0$. Considere a função $f(n)$ da forma $c * n^k$, na qual $c > 0$ e $\in R$, seja a equação de recorrência dada por $T(n) = aT(n/b) + f(n)$ e suponha n uma potência exata de b. Considere ainda que T é uma função que leva números naturais em números reais positivos e que T é assintoticamente não decrescente, Nessas condições, [BRASSARD and BRATLEY, 1996] indicam que a solução dessa recorrência é limitada assintoticamente por

$$T(n) = \theta(n^k) \text{ se } a < b^k; \quad \theta(n^k * \lg n) \text{ se } a = b^k; \quad \theta(n^{\log_b a}) \text{ se } a > b^k$$

Referências

- [BRASSARD and BRATLEY, 1996] BRASSARD, G. and BRATLEY, P. (1996). Fundamentals of Algorithms. Prentice-Hall.

O PDF da lista não tem folha de rosto nem bloco de instruções: começa na Questão 1, com numeração de página a partir de 2.

Sobre esta resolução. Fluxo da seção 5 do `CLAUDE.md`: chão determinístico em Python puro antes de qualquer redação (`verificacao/lista05-chao-deterministico.py`); redação; verificadores adversariais em contexto limpo, quatro por questão e por rodada, cegos ao raciocínio e uns aos outros, com

Bash para conferir as contas; voto assimétrico (4 HOLDS confirma, 2 HOLE refuta); correções; e reverificação da versão corrigida, até a confirmação. **Estado da verificação (14/09/2026).** Todas as cinco questões confirmadas por 4 de 4 verificadores independentes na última rodada: Q1 na 4ª rodada, Q2 na 2ª, Q3 na 3ª, Q4 na 2ª, Q5 na 5ª — 64 verificadores ao todo. Nenhuma correção mudou resposta final nem tocou enunciado; caíram justificativas e contagens (por exemplo, a leitura dupla de A[meio] na Q2 e uma frase introduzida pela própria correção da Q1).

Texto-fonte. Este HTML é gerado por `geradores/gerar_lista05.py` a partir dos arquivos `geradores/dados/lista05/q*-limpo.md` — os mesmos que os verificadores leram —, e o gerador confere que nenhuma linha deles caiu ou duplicou. Registro das correções, com o motivo e quem achou: `geradores/dados/lista05/correcoes-lista05.md`.

Enunciados. Transcritos literalmente do PDF do professor, com os erros de digitação marcados com *[sic]* e nunca corrigidos.

Lista avaliada — IFMG Campus Formiga, 2026/2.